智航Pro二开接口使用说明 v1.0.3

维护记录

本文档面向 用户端(APP)、SDK、算法侧和本体侧联调人员，统一说明 WebSocket 消息、任务状态和网关本体接口。使用时建议先参阅 API Type 总表，再按 type 索引定位至对应章节，查阅请求 / 响应结构、字段说明与状态判定规则。

版本	对应的软件版本	日期	变更说明	备注
v0.0.1		2026-07-15	初版编写	
v1.0.0	Hermes_M_v0.3.1 Hermes_L2_v0.0.7	2026-08-21	新增本体 API	
v1.0.1	Hermes_M_v0.3.1 Hermes_L2_v0.0.7	2026-09-02	• 更新 5011 响应结构及字段定义 • 补充导航 map_type/goal_type 枚举与 SDK 转换说明 • 完善公共消息头 src 来源枚举	
v1.0.2	Hermes_M_v0.3.1 Hermes_L2_v0.0.8	2026-09-03	• 更新路径禁区结构：使用地图坐标多边形 `points`，路径增删改及查询协议同步调整	相关协议: 4023、4024、4028
v1.0.3	Hermes_M_v0.3.2 Hermes_L2_v0.0.8	2026-09-03	• 新增导航避障等级调整协议，返回实际生效等级	相关协议： `4044` 、 `4045`

1. 连接使用说明

1.1 连接模型与建链时序

本节以连接 NVIDIA Orin NX 16G 控制器为例进行说明。

> 尚未部署智航 Pro 版本时，可在 Orin NX 上部署智航 Open 开源版导航功能，相关操作说明参见 https://github.com/zsibot。

智航 Open 由社区共同维护，供 L1 - EDU 客户用于科研场景。如需部署至生产环境，请联系智航产品支持团队获取 Pro 版本授权及相关说明。

1. 笔记本连接本体 Wi-Fi 热点。
   - Wi-Fi 名称：（以 M1 系列设备为例）以 M1 开头、以 SN 号后三位结尾。
   - 初始密码为：12345678。
2. 通过 SSH 连接 Orin NX。

代码块

```
# M1 系列
ssh robot@192.168.168.100
# 密码为：1
```

本体 WebSocket 通过同一端口承载两条逻辑连接：`control` 用于会话建立、控制请求和小数据消息；`bulkdata` 用于大负载数据传输。上述两条连接须归属于同一逻辑客户端 `client_id`。

1.1.1 访问地址

> M1 / L2 的智航地址为 192.168.168.100，对应控制接口连接地址为 `ws://192.168.168.100:10010/control`。

连接	地址	用途
`control`	`ws://host:10010/control`	握手、心跳、控制请求和响应
`bulkdata`	`ws://host:10010/bulkdata`	大负载数据传输

两条连接共用端口 `10010`，通过 URL path 区分连接角色；host 须替换为实际部署的机器人或网关 IP 地址。

1.1.2 逻辑关系

代码块

```
Logical Client (client_id = N)
├── control    -> /control
```

```
3   └── bulkdata  -> /bulkdata
```

`client_id` 由网关在 `1201` 握手应答中分配。`control` 与 `bulkdata` 不可作为两个独立客户端使用，`bulkdata` 须绑定至与 `control` 相同的 `session_id`。

> 💡 Bulkdata 通道主要为部分高频推送数据所使用，主要为1401，1403，1405，1407推送数据

1.1.3 建链约束

1. 必须先建立 `control`，并取得 `1201` 返回的 `session_id` 与 `bulkdata_bind_token`。
2. 建立 `bulkdata` 后，首个数据帧必须发送 `1202` 绑定请求。
3. `1202.data.session_id` 和 `1202.data.token` 必须分别使用 `1201` 返回的会话 ID 和绑定令牌。
4. 只有收到 `1203` 后，`bulkdata` 才视为绑定成功，可以继续传输业务数据。

1.1.4 标准建链时序

1. 客户端连接 `ws://host:10010/control`。
2. 网关主动返回 `1201 kHandShakeAck`，客户端保存 `client_id`、`session_id` 和 `bulkdata_bind_token`。
3. 如需重新获取会话信息，客户端可在 `control` 上发送 `1200 kHandShake`，网关再次返回 `1201`。
4. 客户端连接 `ws://host:10010/bulkdata`。
5. 连接建立后的第一条消息发送 `1202 kBindBulkDataChannel`。
6. 网关返回 `1203 kBindBulkDataChannelAck`，确认绑定成功。
7. 绑定成功后，客户端再发送或接收需要走 `bulkdata` 的业务消息。

1.1.5 建链失败处理

场景	处理
`control` 连接失败	不得建立或继续使用 `bulkdata`，应先重建 `control`。
`bulkdata` 首帧不是 `1202`	网关拒绝绑定，客户端关闭该连接并重新建立 `bulkdata`。
会话 ID 或令牌校验失败	丢弃当前 `bulkdata` 会话，重新获取 `1201` 后再绑定。
未收到 `1203`	不得发送业务数据；按超时策略重试绑定。

`1201` 成功后，客户端须保存 `session_id`、`bulkdata_bind_token` 和 `client_id`；bulkdata 通道建立后须立即发送 `1202`。`1207` 的 `data.modules[].available` 用于判断 SDK、本体桥接模块是否可用；当连续约 3 秒未收到 SDK `1525` 心跳时，gateway 将对应模块置为不可用。

2. 智航 API

API Type 总表

本表按业务域汇总，并按 `type` 编号列出本协议使用的全部消息。请求 `type` 编号通常采用偶数，对应响应为请求
`type + 1`；通知与主动上报采用奇数。具体字段和行为以各章节详细定义为准。

> 💡 下列数据走 control 通道

序号	Type	消息名称	方向	类别	状态 / 说明	操作类型
1	`4001`	地图保存结果通知	ALG → 外部端	通知	建图停止并完成保存收尾后推送	推送
2	`4012`	获取地图列表	外部端 → ALG	请求	地图查询	查询
3	`4013`	获取地图列表响应	ALG → 外部端	响应	对应 `4012`	查询响应
4	`4014`	修改地图目录名	外部端 → ALG	请求	地图管理	控制
5	`4015`	修改地图目录名响应	ALG → 外部端	响应	对应 `4014`	控制响应
6	`4016`	删除地图	外部端 → ALG	请求	地图管理	控制
7	`4017`	删除地图响应	ALG → 外部端	响应	对应 `4016`	控制响应
8	`4018`	设置返航回充点位	外部端 → ALG	请求	地图管理	控制
9	`4019`	设置返航回充点位响应	ALG → 外部端	响应	对应 `4018`	控制响应
10	`4021`	故障码上报	ALG → 外部端	上报	事件驱动的故障列表	推送
11	`4022`	根据地图获取路径信息	外部端 → ALG	请求	路径查询	查询
12	`4023`	获取路径信息响应	ALG → 外部端	响应	对应 `4022`	查询响应

13	`4024`	添加路径	外部端 → ALG	请求	路径管理	控制
14	`4025`	添加路径响应	ALG → 外部端	响应	对应 `4024`	控制响应
15	`4026`	删除路径	外部端 → ALG	请求	路径管理	控制
16	`4027`	删除路径响应	ALG → 外部端	响应	对应 `4026`	控制响应
17	`4028`	修改路径	外部端 → ALG	请求	路径管理	控制
18	`4029`	修改路径响应	ALG → 外部端	响应	对应 `4028`	控制响应
19	`4030`	定位请求	外部端 → ALG	请求	载图或重定位	控制
20	`4031`	定位响应	ALG → 外部端	响应	对应 `4030`	控制响应
21	`4033`	定位状态上报	ALG → 外部端	上报	**已废弃**，已折入 `5009`	—
22	`4035`	建图位姿及里程上报	ALG → 外部端	上报	跟随 SLAM 状态流推送	推送
23	`4036`	删除返航回充点位	外部端 → ALG	请求	地图管理	控制
24	`4037`	删除返航回充点位响应	ALG → 外部端	响应	对应 `4036`	控制响应
25	`4038`	获取机器人时间	外部端 → ALG	请求	查询机器当前本地时间与 Unix 毫秒	查询
26	`4039`	获取机器人时间响应	ALG → 外部端	响应	对应 `4038`	查询响应
27	`5000`	创建任务	外部端 → ALG	请求	建图、导航、回充、出桩、巡检等任务	控制
28	`5001`	创建任务响应	ALG → 外部端	响应	对应 `5000`	控制响应
29	`5002`	取消任务	外部端 → ALG	请求	按 `task_id` 取消	控制
30	`5003`	取消任务响应	ALG → 外部端	响应	对应 `5002`	控制响应
31	`5004`	暂停任务	外部端 → ALG	请求	按 `task_id` 暂停	控制
32	`5005`	暂停任务响应	ALG → 外部端	响应	对应 `5004`	控制响应
33	`5006`	恢复任务	外部端 → ALG	请求	按 `task_id` 恢复	控制
34	`5007`	恢复任务响应	ALG → 外部端	响应	对应 `5006`	控制响应

35	`5009`	任务及定位状态上报	ALG → 外部端	上报	V2 单帧快照；按配置周期心跳（默认 500 ms），空闲时 `current_task=null`	推送
36	`5010`	查询巡检任务记录（暂不对外提供）	外部端 → ALG	请求	巡检记录查询	查询
37	`5011`	查询巡检任务记录响应（暂不对外提供）	ALG → 外部端	响应	对应 `5010`	查询响应
38	`5014`	返回充电点	外部端 → ALG	请求	新建返回充电点任务	控制
41	`5015`	返回充电点响应	ALG → 外部端	响应	对应 `5014`	控制响应
43	`5019`	回充/出桩底层结果通知	ALG → 外部端	通知	`5000 kind=3/5` 或 `5014` 的底层结果：充上电 / 失败 / 满电保持	推送
44	`5020`	删除巡检任务记录（暂不对外提供）	外部端 → ALG	请求	按 `task_id` 列表批量删除	控制
45	`5021`	删除巡检任务记录响应（暂不对外提供）	ALG → 外部端	响应	对应 `5020`	控制响应
46	`4040`	手动录包控制	外部端 → ALG	请求	开始/停止录制	控制
47	`4041`	手动录包控制响应	ALG → 外部端	响应	对应 `4040`	控制响应
48	`4042`	手动录包状态查询	外部端 → ALG	请求	获取录制状态	查询
49	`4043`	手动录包状态查询响应	ALG → 外部端	响应	对应 `4042`	查询响应

📝 **v1.0.3 新增 Type：** `4044/4045` 为导航避障等级调整请求与响应。

序号	Type	消息名称	方向	类别	状态 / 说明	操作类型
50	`4044`	设置导航避障等级	外部端 → ALG	请求	调整当前导航任务的避障能力	控制

51	`4045`	设置导航避障等级响应	ALG → 外部端	响应	对应 `4044`，返回实际生效等级	控制响应

消息推送补充索引

以下实时数据推送用于地图显示和运行态监控，详细字段与数据结构见 2.3.2 消息推送。

> 💡 下列数据走 Bulkdata 通道

type	名称	数据结构	类别
`1401`	`kCurrentPose` 当前机器人位姿	`pose`：位置与姿态	实时位姿
`1403`	`kSlamPath` 建图轨迹	`poses[]`：轨迹点数组	轨迹推送
`1405`	`kLocalPath` 局部规划路径	`poses[]`：路径点数组	路径推送
`1407`	`kGlobalPath` 全局规划路径	`poses[]`：路径点数组	路径推送

2.1 API 使用约定

传输方式。所有请求、响应、通知和推送均通过 WebSocket 文本消息传输，消息统一由 `head` 和 `data` 两部分组成。

请求与响应。请求消息的 `src=1`，算法响应的 `src=2`；使用 `msg_id` 关联同一请求的响应。客户端应先校验 `head.type` 和 `msg_id`，再解析 `data`。

通知与推送。通知、状态和实时数据由算法主动发送，不与请求建立一一对应关系；客户端按 `head.type` 分发，并分别维护事件、状态和实时数据。

错误处理。响应优先判断 `data.code`：`0` 表示受理或处理成功，非 `0` 表示失败；再结合 `data.msg` 和业务字段展示结果。控制类响应仅表示请求受理，任务最终结果以状态推送或结果通知为准。

本章按 API 参考格式组织，覆盖公共消息头、事件通知、地图与路径、定位、任务控制、状态上报和巡检记录。每个接口均以 type 为索引，说明方向、请求或推送条件、data 结构和客户端处理方式。

2.2 公共消息头

2.2.1 消息头定义

代码块

```
{
    "type": 4001,
```

```
    "time": 1757310525610,
    "src": 1,
    "msg_id": 12345
}
```

字段	类型	说明
type	number	消息类型
time	number	消息时间戳，单位毫秒
src	number	消息来源枚举；外部请求按发送方使用 `1` / `3` / `6`，ALG 响应、通知、推送、上报固定为 `2`
msg_id	number	消息 ID，用于请求与响应关联

src	来源	说明
1	app	App
2	alg	算法
3	cloud	云端
4	machine_body	机器人本体
5	gateway	Gateway
6	sdk	SDK

2.2.2 通用响应字段

字段	类型	说明
code	number	响应码，`0` 表示成功，非 `0` 表示失败
msg	string	响应说明

2.3 公共 API

公共接口用于承载不直接归属于建图、定位、导航控制流程的通用能力。

2.3.1 事件通知

2.2.1.1 地图保存结果通知 `4001`

建图任务停止并完成保存收尾后，算法侧将主动下发一次本通知，**无论保存成功或失败均会推送**，客户端须根据 `data.code` 判定处理结果。

保存成功：

```
代码块
1  {
2      "head": {
3          "type": 4001,
4          "time": 1757310525610,
5          "src": 2,
6          "msg_id": 12345
7      },
8      "data": {
9          "code": 0,
10         "msg": "ok"
11     }
12 }
```

保存失败（示例：保存超时）：

```
代码块
1  {
2      "head": {
3          "type": 4001,
4          "time": 1757310525610,
5          "src": 2,
6          "msg_id": 12346
7      },
8      "data": {
9          "code": 13980,
10         "msg": "mapping: save map timed out"
11     }
12 }
```

字段	类型	说明
data.code	number	`0` 表示地图保存成功；非 `0` 表示失败，取值为错误码表中的对应码（十进制）

data.msg	string	结果说明：成功为 `ok`，失败为对应错误码的英文描述

2.2.1.2 回充 / 出桩底层结果通知 `5019`

`5019` 表示底层充电 / 出桩处理结果已确定，不表示请求是否受理。请求是否受理以 `5001` 或 `5015` 为准，导航至充电点等过程状态以 `5009` 为准。底层结果为取消 / 主动停止时不推送 `5019`。

`data.status` 的取值规则按 `task_type` 区分（回充与出桩分别适用独立的状态判定逻辑）：

task_type	`1` 成功	`2` 失败	`3` 满电保持
`recharge` / `return_to_charge`	**确认进入充电状态** —— 两节电池任一节 `power_supply_status == 1`（CHARGING）并持续 3 秒	底层 arc 报 `FAILURE` / `STOPPED`，或状态流中断触发保活超时	底层 `phase == recharge_completed`
`undock`	底层 arc 报 `SUCCESS`	同上	不适用

推送时机独立于任务终结状态。 回充任务在充上电之后**不会立即结束**（大状态持续 `1` 运行中，子状态转 `43` 充电中 / `45` 满电保持），所以：

- `status = 1` 在**确认进入充电状态时**推送，此时任务仍在运行；
- `status = 3` 在底层报 `recharge_completed` 时推送一次（不重复推）；
- `status = 2` 在任务失败终结时推送。

同一回充任务可能依次收到 `1` 和 `3` 两帧。

回充成功（`5000 kind=3`）：

代码块

```
{
    "head": {
        "type": 5019,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12352
    },
    "data": {
        "task_id": "xxxxxxx",
        "task_type": "recharge",
        "kind": 3,
        "status": 3,
```

```
        "code": 0,
        "msg": "recharge succeeded"
    }
}
```

出桩失败（`5000 kind=5`）：

代码块

```
{
    "head": {
        "type": 5019,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12353
    },
    "data": {
        "task_id": "xxxxxxx",
        "task_type": "undock",
        "kind": 5,
        "status": 4,
        "code": 14000,
        "msg": "undock failed"
    }
}
```

`5014` 底层充电结果的 `task_type` 为 `return_to_charge`，`kind` 为 `null`，其余字段含义相同。

字段	类型	说明
data.task_id	string	任务 ID；`5014` 为 `5015` 返回的任务 ID
data.task_type	string	`recharge` 回充、`undock` 出桩、`return_to_charge` 表示 `5014` 的底层回充
data.kind	number/null	`3` 回充、`5` 出桩；`5014` 固定为 `null`
data.status	number	`1` 成功、`2` 失败、`3` 满电保持。判据随 `task_type` 而异，见下方说明
data.code	number	成功为 `0`；失败为底层 / 算法错误码
data.msg	string	结果说明

2.3.2 消息推送

消息推送接口由算法侧主动向 用户端（app）发送实时数据。本节按推送类型、消息结构、触发条件与客户端处理方式组织，客户端须按 `head.type` 分发，并分别维护事件、状态和路径数据。

2.2.2.1 推送类型总览

本节先行给出推送总览，再说明 `4021`、`4035` 与 `1401/1403/1405/1407` 的数据结构。`5009` 的任务与定位状态在 §2.6 节单独定义；本节仅说明其与其他实时推送的关系。

推送类别	type	方向 / 触发条件	data 重点	客户端处理
故障码事件	`4021`	故障变化时推送	`items[]`：故障码、描述、等级	告警中心去重、展示和恢复
建图遥测	`4035`	建图运行及保存收尾阶段推送	`slam_odom_pose`、`mileage`、`remain_mileage`	显示建图位姿和里程
任务与定位状态	`5009`	默认每 500 ms 心跳，状态变化即时补推	`status`、`task`、`localization`	更新任务、定位和进度状态
当前位姿	`1401`	实时推送	`pose`：位置与姿态	更新机器人当前位置
建图轨迹	`1403`	建图过程中持续推送	`poses[]`：历史轨迹点	绘制建图轨迹
局部规划路径	`1405`	规划更新时推送	`poses[]`：局部路径点	绘制局部规划路径
全局规划路径	`1407`	规划更新时推送	`poses[]`：全局路径点	绘制全局规划路径

2.2.2.2 故障码上报 `4021`

代码块

```
{
    "head": {
        "type": 4021,
        "time": 1763697492747,
        "src": 2,
        "msg_id": 1
    },
    "data": {
        "items": [
            {
                "code": 13330,
                "description": "navigation blocked",
```

```
            "severity": 0
        },
        {
            "code": 13331,
            "description": "lidar disconnected",
            "severity": 1
        }
    ]
  }
}
```

2.2.2.3 建图位姿与里程推送 `4035`

建图任务处于运行阶段或停止保存收尾阶段时，算法侧将随建图状态变化下发 `4035` 。建图任务开始前、任务成功 / 失败终结后均不推送；发送频率随建图状态更新频率自适应。

代码块

```
{
    "head": {
        "type": 4035,
        "time": 1763697492747,
        "src": 2,
        "msg_id": 1
    },
    "data": {
        "slam_odom_pose": {
          "position":    { "x": 1.0, "y": 2.0, "z": 0.0 },
          "orientation": { "x": 0.0, "y": 0.0, "z": 0.0, "w": 1.0 }
        },
        "mileage": 123.4,
        "remain_mileage": 10.2
    }
}
```

字段	类型	说明
data.slam_odom_pose.position	object	SLAM 里程位姿的位置 `x/y/z` ，单位米
data.slam_odom_pose.orientation	object	SLAM 里程位姿四元数 `x/y/z/w`
data.mileage	number	本次建图累计行驶里程，单位米
data.remain_mileage	number	底层估算的剩余里程，单位米

> `4035` 已从 `5009` 中拆分。`5009` 只负责建图任务主状态和子状态，例如 `Mapping:Running`、`Mapping:Save`、成功或失败；不再携带位姿与里程字段。

2.2.2.4 实时位姿与路径数据结构

type	data 字段	字段说明
`1401`	`pose.position`、`pose.orientation`	当前位置 `x/y/z`；朝向四元数 `x/y/z/w`
`1403`	`poses[]`	建图历史轨迹点数组，每个元素包含位置和朝向
`1405`	`poses[]`	局部规划路径点数组，每个元素包含位置和朝向
`1407`	`poses[]`	全局规划路径点数组，每个元素包含位置和朝向

四类消息均采用统一的 `head` 结构；路径类消息使用 `poses[]`，位姿类消息使用位置和姿态字段。客户端须按 `type` 分别存储，避免不同数据模型相互覆盖。

当前位姿上报 1401

归属：智航交互 → 消息推送。**方向：**本体 → 用户端(APP)。**触发：**实时推送。

字段	类型	说明
`data.pose.position`	object	`x/y/z`，单位 m
`data.pose.orientation`	object	四元数 `x/y/z/w`
`data.twist.linear`	object	线速度，单位 m/s
`data.twist.angular`	object	角速度，单位 rad/s

代码块

```
{"head":{"type":1401,"src":5,"time":1785315335685},"data":{"pose":{"position":{"x":0,"y":0,"z":0},"orientation":{"x":0,"y":0,"z":0,"w":1}},"twist":{"linear":{"x":0,"y":0,"z":0},"angular":{"x":0,"y":0,"z":0}}}}
```

建图轨迹上报 1403

归属：智航交互 → 消息推送。**方向：**本体 → 用户端(APP)。**触发：**建图过程中持续推送。

字段	类型	说明

`data.header`	object	轨迹数据的时间戳和坐标系标识
`data.poses[]`	array	历史轨迹点；每个元素包含 `pose.position` 和 `pose.orientation`

代码块

```
{"head":{"type":1403,"src":5,"time":1785345771185},"data":{"header":{"frame_id":"map","stamp":{"sec":0,"nanosec":0}},"poses":[{"pose":{"position":{"x":0,"y":0,"z":0},"orientation":{"x":0,"y":0,"z":0,"w":1}}}]}}
```

局部规划路径 1405

归属：智航交互 → 消息推送。**方向：**本体 → 用户端(APP)。**触发：**局部规划更新时推送。数据结构与 `1403` 一致， `poses[]` 表示局部路径点。

全局规划路径 1407

归属：智航交互 → 消息推送。**方向：**本体 → 用户端(APP)。**触发：**全局规划更新时推送。数据结构与 `1403` 一致， `poses[]` 表示全局路径点。

2.2.2.5 客户端接入说明

1. **按 `head.type` 分发。**不同 type 使用对应的数据模型和处理逻辑。
2. **按消息时间戳更新。**同一 type 仅接受较新的 `head.time` ，旧帧不得覆盖新数据。
3. **区分响应与最终状态。**控制请求响应只表示请求已受理；任务完成、失败或取消以状态推送或结果通知为准。

2.3.3 路径管理（巡检任务使用，暂不对外使用）

2.2.3.1 根据地图获取路径信息 `4022`

代码块

```
{
    "head": {
        "type": 4022,
        "time": 1763020729847,
        "src": 1,
        "msg_id": 1
    },
    "data": {
        "map_id": "xxxxx"
    }
}
```

2.2.3.2 根据地图获取路径信息响应 `4023`

示例


```
{
    "head": {
        "type": 4023,
        "time": 1763020729847,
        "src": 2,
        "msg_id": 1
    },
    "data": {
        "code": 0,
        "msg": "ok",
        "map_id": "map_id",
        "path_names": [
            "path1"
        ],
        "paths": {
            "path1": [
                [
                    "a",
                    {
                        "position": {
                            "x": 0,
                            "y": 0,
                            "z": 0
                        },
                        "orientation": {
                            "x": 0,
                            "y": 0,
                            "z": 0,
                            "w": 1
                        },
                        "properties": {
                            "type": 1,
                            "tasks": [
                                0,
                                1
                            ]
                        }
                    }
                ]
            ]
        },
        "fences": {
            "path1": [
                {
```

```
                    "id": 1768201485449,
                    "points": [
                        {
                            "x": 19.326860294653972,
                            "y": -7.948306378962686
                        },
                        {
                            "x": 26.068063921759357,
                            "y": 2.3231087941233106
                        },
                        {
                            "x": 17.37213970534604,
                            "y": 8.030306378962694
                        },
                        {
                            "x": 10.630936078240657,
                            "y": -2.2411087941233028
                        }
                    ]
                },
                {
                    "id": 2,
                    "points": [
                        {
                            "x": 19.326860294653972,
                            "y": -7.948306378962686
                        },
                        {
                            "x": 26.068063921759357,
                            "y": 2.3231087941233106
                        },
                        {
                            "x": 17.37213970534604,
                            "y": 8.030306378962694
                        },
                        {
                            "x": 10.630936078240657,
                            "y": -2.2411087941233028
                        }
                    ]
                }
            ]
        },
        "skipped_count": 0
    }
}
```

2.2.3.3 添加路径 `4024`

示例


```
{
    "head": {
        "type": 4024,
        "time": 1763025609915,
        "src": 1,
        "msg_id": 1
    },
    "data": {
        "add_nav_path": [
            "map_id",
            "path_id",
            [
                [
                    "a",
                    {
                        "position": {
                            "x": 0,
                            "y": 0,
                            "z": 0
                        },
                        "orientation": {
                            "x": 0,
                            "y": 0,
                            "z": 0,
                            "w": 1
                        },
                        "properties": {
                            "type": 1,
                            "tasks": [
                                0,
                                1
                            ]
                        }
                    }
                ]
            ],
            [
                {
                    "id": 1768201485449,
                    "points": [
                        {
                            "x": 19.326860294653972,
                            "y": -7.948306378962686
                        },
```

```
                        {
                            "x": 26.068063921759357,
                            "y": 2.3231087941233106
                        },
                        {
                            "x": 17.37213970534604,
                            "y": 8.030306378962694
                        },
                        {
                            "x": 10.630936078240657,
                            "y": -2.2411087941233028
                        }
                    ]
                },
                {
                    "id": 2,
                    "points": [
                        {
                            "x": 19.326860294653972,
                            "y": -7.948306378962686
                        },
                        {
                            "x": 26.068063921759357,
                            "y": 2.3231087941233106
                        },
                        {
                            "x": 17.37213970534604,
                            "y": 8.030306378962694
                        },
                        {
                            "x": 10.630936078240657,
                            "y": -2.2411087941233028
                        }
                    ]
                }
            ]
        ]
    }
}
```

> `add_nav_path` 元组：`[map_id, path_id, points, forbidden_zones]`。`forbidden_zones` 为禁区数组，每个元素仅包含 `id` 和地图坐标系下的多边形 `points`；无禁区时传空数组。旧的 `center`、`width`、`height`、`rotation` 结构不再兼容。

2.2.3.4 删除路径 `4026`

代码块

```
{
    "head": {
        "type": 4026,
        "time": 1763020634597,
        "src": 1,
        "msg_id": 1
    },
    "data": {
        "remove_nav_path": [
            ["map_id_1", "path_id_1"],
            ["map_id_1", "path_id_2"]
        ]
    }
}
```

2.2.3.5 修改路径 `4028`

示例

```
{
    "head": {
        "type": 4028,
        "time": 1763025784597,
        "src": 1,
        "msg_id": 1
    },
    "data": {
        "modify_nav_path": [
            "map1",
            "old_path",
            "new_path",
            [
                [
                    "a",
                    {
                        "position": {
                            "x": 0,
                            "y": 0,
                            "z": 0
                        },
                        "orientation": {
                            "x": 0,
                            "y": 0,
                            "z": 0,
                            "w": 1
                        },
```

```
                    "properties": {
                        "type": 1,
                        "tasks": [
                            0,
                            1
                        ]
                    }
                }
            ]
        ],
        [
            {
                "id": 1768201485449,
                "points": [
                    {
                        "x": 19.326860294653972,
                        "y": -7.948306378962686
                    },
                    {
                        "x": 26.068063921759357,
                        "y": 2.3231087941233106
                    },
                    {
                        "x": 17.37213970534604,
                        "y": 8.030306378962694
                    },
                    {
                        "x": 10.630936078240657,
                        "y": -2.2411087941233028
                    }
                ]
            },
            {
                "id": 2,
                "points": [
                    {
                        "x": 19.326860294653972,
                        "y": -7.948306378962686
                    },
                    {
                        "x": 26.068063921759357,
                        "y": 2.3231087941233106
                    },
                    {
                        "x": 17.37213970534604,
                        "y": 8.030306378962694
                    },
```

```
                        {
                            "x": 10.630936078240657,
                            "y": -2.2411087941233028
                        }
                    ]
                }
            ]
        ]
    }
}
```

`modify_nav_path` 元组：`[map_id, old_path, new_path, points, forbidden_zones]`。`forbidden_zones` 为禁区数组，每个元素仅包含 `id` 和地图坐标系下的多边形 `points`；无禁区时传空数组。旧的 `center`、`width`、`height`、`rotation` 结构不再兼容。

2.2.3.6 路径增删改响应 `4025/4027/4029`

代码块

```
{
    "head": {
        "type": 4025,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

`4025`、`4027`、`4029` 分别对应路径的添加、删除、修改响应。

2.2.3.7 路径点结构与 `properties`

`4022/4023/4024/4028` 中的每个路径点均为二元组 `[点名, {position, orientation, properties?}]`，`properties` 描述该点的巡检语义：

字段	类型	说明
`type`	number	点类型：`0` 途径点（只路过，不作业）/ `1` 作业点（到点后依次执行 `tasks`）

`tasks`	number[]	**作业动作类型**序列，按数组顺序依次执行。取值见下表；空数组 / `null` / 省略 = 无作业动作

`tasks` 取值	动作	说明
`0`	等待	在该点停留一段固定时长（当前实现 2 秒）
`1`	拍照	调用上装拍照（`/capture_image`），异步 + 超时；失败只记录，不中断巡检

约束（算法侧按此校验，违反即忽略并告警，但**不会**让整条路径加载失败）：

- `tasks` 是**动作类型**，不是"作业项 ID"——**当前取值闭集为 `{0, 1}`**，其它数字一律视为非法并丢弃；
- `type=0` 的途径点**不执行任何动作**：即便写了 `tasks` 也会被忽略（要执行请把该点标为 `type=1`）；
- 同一动作可重复出现（如 `[0, 1, 0]` = 等待→拍照→等待）。

> 与 `5000` 巡检任务内联点位（§4.1.7 的 `goals[].actions[].type`）取值口径一致，两处都是 `0` 等待 / `1` 拍照。

2.3.4 地图相关

2.2.4.1 获取地图列表 `4012`

代码块

```
{
    "head": {
        "type": 4012,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": null
}
```

2.2.4.2 地图列表响应 `4013`

代码块

```
{
    "head": {
```

```
        "type": 4013,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok",
        "map_dir_prefix": "/ota/alg_data/map",
        "map_path_list": [
            {
                "path": "/ota/alg_data/map/history_map/xxxxx",
                "map_route_exist": true,
                "charge_point_exist": false,
            },
            {
                "path": "/ota/alg_data/map/history_map/yyyyy",
                "map_route_exist": false,
                "charge_point_exist": true,
            }
        ]
    }
}
```

`map_path_list` 中每一项对应一个有效地图目录：

字段	类型	说明
map_dir_prefix	string	配置的地图数据根目录，即 `map_root_dir`；例如 `/ota/alg_data/map`
path	string	地图目录的真实绝对路径，格式为 `<map_dir_prefix>/history_map/<地图目录名>`
map_route_exist	boolean	该目录下是否存在普通文件 `map.geojson`，供 用户端(APP) 判断该地图是否支持路网导航
charge_point_exist	boolean	该地图是否已设置**可用的**回充点位（`4018` 写入的 `charge_point.json`）。判据不只是"文件存在"，还要求能解析出合法位姿：各分量为有限数、且姿态四元数非零。文件缺失、字段残缺、数值非法一律为 `false`

> `charge_point_exist` 与巡检 `end_task=2`、返回充电点 `5014` 实际取点用的是**同一套校验**。因此该字段为 `true` 时，返充功能必定有目标点可用；为 `false` 时，用户端(APP) 应置灰返充入口，否则用户点击会收到 `0x3672 ChargePointNotSet`。

2.2.4.3 修改地图目录名 `4014`

代码块

```
{
    "head": {
        "type": 4014,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "from": "old_name",
        "to": "new_name"
    }
}
```

2.2.4.4 修改地图目录名响应 `4015`

代码块

```
{
    "head": {
        "type": 4015,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

2.2.4.5 删除地图 `4016`

代码块

```
{
    "head": {
        "type": 4016,
        "time": 1763020729847,
        "src": 1,
        "msg_id": 1
    },
    "data": {
```

```
        "map_id": [
            "id1",
            "id2"
        ]
    }
}
```

2.2.4.6 删除地图响应 `4017`

代码块

```
{
    "head": {
        "type": 4017,
        "time": 1763020729847,
        "src": 2,
        "msg_id": 1
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

2.2.4.7 设置返航回充点位 `4018`

代码块

```
{
  "head": {
    "type": 4018,
    "time": 1763020729847,
    "src": 1,
    "msg_id": 1
  },
  "data": {
    "map_id": "xxx",
    "pose": {
      "position": {
        "x": -0.36639437079429626,
        "y": -0.0043663992546498775,
        "z": 0.014214999973773956
      },
      "orientation": {
        "x": 0.016655222306478958,
        "y": 0.012954059767979796,
```

```
        "z": 0.0003800384037053115,
        "w": 0.9997773009406951
      }
    }
  }
}
```

字段	类型	说明
head.type	number	固定为 `4018`
data.map_id	string	回充点所属地图 ID，地图必须已经存在
data.pose.position	object	回充导航点位置，包含有限数值 `x/y/z`，单位米
data.pose.orientation	object	回充导航点姿态四元数，包含有限数值 `x/y/z/w`，且不能为零四元数

算法侧校验通过后原子覆盖地图目录下的 `charge_point.json`。文件内容为 `{"pose": {"position": {...}, "orientation": {...}}}`；重复设置同一地图的回充点是幂等覆盖操作。

2.2.4.8 设置返航回充点位响应 `4019`

代码块

```
{
  "head": {
    "type": 4019,
    "time": 1763020729847,
    "src": 2,
    "msg_id": 1
  },
  "data": {
    "code": 0,
    "msg": "ok"
  }
}
```

2.2.4.9 删除返航回充点位 `4036`

代码块

```
{
```

```
  "head": {
    "type": 4036,
    "time": 1763020729847,
    "src": 1,
    "msg_id": 1
  },
  "data": {
    "map_id": "xxx"
  }
}
```

字段	类型	说明
head.type	number	固定为 `4036`
data.map_id	string	要删除回充点的地图 ID，地图必须已经存在

算法侧删除该地图目录下的 `charge_point.json`。删除采用幂等语义：地图存在但回充点文件本就不存在时仍返回成功。删除成功后再次获取地图列表（`4012/4013`），对应地图的 `charge_point_exist` 为 `false`。已经启动的巡检或返充任务可能已把点位读入内存，本操作只保证后续新建任务不再读取该回充点，不中止正在执行的任务。

> 编号说明：地图管理的下一个偶数 `4020` 所对应的响应号 `4021` 已用于故障上报；`4032/4034` 的响应号也分别与现有上报冲突，因此本协议使用 `4036/4037`。

2.2.4.10 删除返航回充点位响应 `4037`

代码块

```
{
  "head": {
    "type": 4037,
    "time": 1763020729847,
    "src": 2,
    "msg_id": 1
  },
  "data": {
    "code": 0,
    "msg": "ok"
  }
}
```

2.3.5 获取机器人时间 `4038/4039`

用于获取机器人当前系统时间。请求无需参数，算法侧收到请求时只采样一次系统时钟，并由同一个采样值生成本地时间字符串和 Unix epoch 毫秒，避免两个字段跨毫秒边界。

请求 `4038`

代码块

```
{
  "head": {
    "type": 4038,
    "time": 1786344054700,
    "src": 1,
    "msg_id": 12345
  },
  "data": {}
}
```

响应 `4039`

代码块

```
{
  "head": {
    "type": 4039,
    "time": 1786344054741,
    "src": 2,
    "msg_id": 12345
  },
  "data": {
    "code": 0,
    "msg": "ok",
    "timestamp_fmt": "2026-08-10 14:40:54.741",
    "timestamp_ms": "1786344054741"
  }
}
```

字段	类型	说明
head.type	number	请求固定为 `4038`，响应固定为 `4039`
head.msg_id	number	响应回显请求消息 ID
data.timestamp_fmt	string	机器人本地时间，格式 `yyyy-MM-dd HH:mm:ss.SSS`，受机器人时区配置影响

data.timestamp_ms	string	Unix epoch 毫秒的十进制字符串；字段名和类型与老 Rust 版保持一致

2.3.6 设置导航避障等级 `4044/4045`

> 💡 本接口采用新的数字 Type 协议，只复用统一消息头和 `code/msg` 规则；不复用旧版字符串 WebSocket 协议中的 `req_func`、`status`、`result` 字段。

请求 `4044`

设置避障等级请求

```
{
  "head": {
    "type": 4044,
    "time": 1788446400000,
    "src": 1,
    "msg_id": 12345
  },
  "data": {
    "level": 30
  }
}
```

`data.level`	名称	行为
`10`	`none`	关闭障碍物感知层和碰撞停止，机器人不避障
`20`	`stop_only`	不绕障，但保留碰撞停止
`30`	`perceive`	感知并停车等待，不执行局部绕障
`40`	`full`	完整避障：感知、停车和绕障；默认等级

`level` 必须是上述四个整数之一。字段缺失、字符串、小数或其它数值均返回 `13898 (0x364A, BadRequest)`，且不会调用底层服务。

成功响应 `4045`

设置成功

```
{
```

```
  "head": {
    "type": 4045,
    "time": 1788446400042,
    "src": 2,
    "msg_id": 12345
  },
  "data": {
    "code": 0,
    "msg": "ok",
    "requested_level": 30,
    "effective_level": 30,
    "reason": "applied"
  }
}
```

底层拒绝或应用失败时，响应仍返回 `requested_level`、`effective_level` 和稳定的 `reason`。用户端(APP)必须以 `effective_level` 展示当前真实状态，不能假设请求等级已经生效。

设置被部署策略拒绝

```
{
  "head": {
    "type": 4045,
    "time": 1788446400042,
    "src": 2,
    "msg_id": 12345
  },
  "data": {
    "code": 13915,
    "msg": "obstacle avoidance level rejected",
    "requested_level": 10,
    "effective_level": 40,
    "reason": "not_permitted"
  }
}
```

reason	code	含义
`applied`	`0`	设置成功
`invalid_level`	`13915`	底层判定等级非法
`not_supported`	`13915`	当前导航版本不支持该等级

`not_permitted`	`13915`	低于部署允许的最低等级
`partial_failure`	`13968`	部分导航组件更新失败，底层已回退到 `full`
`timeout`	`13968`	底层内部应用超时并回退
`undefined_failure` / `service_error`	`13968`	底层未定义失败或未知结果

服务未就绪或调用异常返回 `13966 (0x368E)`；服务调用超过 3000 ms 返回 `13967 (0x368F)`。这两类传输失败没有可信的底层应答，因此响应不带 `requested_level/effective_level/reason`。

> ❗ **任务级生效：**导航状态每次进入 `STANDBY` 都会恢复为 `40 (full)`。若新任务需要较低等级，用户端(APP) 应在该任务启动后重新发送 `4044`，不能把上一次成功响应当作跨任务持久配置。

2.4 定位 API

2.4.1 定位请求 `4030`

代码块

```
{
    "head": {
        "type": 4030,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "op": 1,
        "map_path": " ",
        "initial_pose": {
            "position": {"x": 0.0, "y": 0.0, "z": 0.0},
            "orientation": {"x": 0.0, "y": 0.0, "z": 0.0, "w": 1.0}
            // 若 orientation 为0 0 0 0,则SLAM 默认读取上一份位姿文件，使用完整重定位能力
        }
    }
}
```

其中，`op` 标识定位操作类型：`1` 为 `loadmap`（加载地图）。`map_path` 在 `op=1` 时表示待加载的地图路径。

`initial_pose` 为起始位姿，结构与 `geometry_msgs/Pose` 对齐。不传 `initial_pose` 时保持原有定位逻辑；底层等价使用 `0 0 0 0 0 0 0`，由 SLAM 默认读取上一份位姿文件。

2.4.2 定位响应 `4031`

代码块

```
{
    "head": {
        "type": 4031,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "op": 1,
        "code": 0,
        "msg": "ok"
    }
}
```

2.5 任务 API

任务接口用于创建、取消、暂停、恢复任务，以及接收任务状态上报。

2.5.1 创建任务 5000/5001

2.5.1.1 通用请求示例 `5000`

代码块

```
{
    "head": {
        "type": 5000,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "kind": 1,
        "name": "xxxxx",
        "schedule": 0,
        "execute_at": "1757310525610",
        "priority": 0,
        "record": false,
        "params": {}
    }
```

```
17  }
```

字段	类型	说明
head.type	number	消息类型，固定为 `5000`，表示创建任务请求
head.time	number	请求时间戳，单位毫秒
head.src	number	消息来源，App 侧固定为 `1`
head.msg_id	number	消息 ID，用于请求与响应关联
data.kind	number	任务类型，取值见下方 `kind` 枚举
data.name	string	任务名称
data.schedule	number	调度方式，当前仅支持即时任务
data.execute_at	string	预约任务执行时间，当前为预留字段
data.priority	number	优先级，当前为预留字段
data.record	boolean	是否生成任务记录；建议仅巡检任务开启，当前仅巡检任务支持
data.params	object	任务参数，不同任务类型对应不同参数结构；无参数时传 `{}`

kind	任务类型	说明
1	mapping	建图任务
2	navigation	导航任务
3	recharge	无图回充任务（不含返航）
~~4~~	~~patrol~~	~~巡检任务（暂不对外开放）~~
5	undock	出桩任务（不含返航）

schedule	说明
0	即时任务，当前支持
1	预约任务，暂不支持，预留

2.5.1.2 响应示例 `5001`

代码块

```
{
    "head": {
        "type": 5001,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "success",
        "task_id": "018f2f5d-1c20-7a5b-a801-123456789001"
    }
}
```

字段	类型	说明
head.type	number	消息类型，固定为 `5001`，表示创建任务响应
head.time	number	响应时间戳，单位毫秒
head.src	number	消息来源，算法侧固定为 `2`
head.msg_id	number	消息 ID，与创建任务请求中的 `msg_id` 对应
data.code	number	响应码，`0` 表示创建成功，非 `0` 表示创建失败
data.msg	string	响应说明
data.task_id	string	任务 ID，创建成功后返回

2.5.1.3 建图任务示例 `5000`

代码块

```
{
    "head": {
        "type": 5000,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
```

```
        "kind": 1,
        "name": "xxxxx",
        "schedule": 0,
        "execute_at": "1757310525610",
        "priority": 0,
        "record": false,
        "params": {
            "map_type": 0
        }
    }
}
```

建图任务固定使用 `data.kind = 1`，任务参数放在 `data.params` 中。

字段	类型	说明
data.params.map_type	number	建图类型，取值见下方 `map_type` 枚举

map_type	说明
0	室外建图
2	室内建图

2.5.1.4 导航任务示例 `5000`

代码块

```
{
    "head": {
        "type": 5000,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "kind": 2,
        "name": "xxxxx",
        "schedule": 0,
        "execute_at": "1757310525610",
        "priority": 0,
        "record": false,
        "params": {
            "map_path": "xxxxx",
```

```
            "need_locate": true,
            "map_type": 1,
            "goal_type": 1,
            "fence_path": "xxxxx",
            "goals": [
                {
                    "position": {
                        "x": 0,
                        "y": 0,
                        "z": 0
                    },
                    "orientation": {
                        "x": 0,
                        "y": 0,
                        "z": 0,
                        "w": 1
                    }
                }
            ],
            "speed_limit": {
                "vx": 0.5,
                "vy": 0,
                "vz": 1.5
            },
            "goal_tolerance": {
                "x": 0.25,
                "y": 0,
                "theta": 0.25
            }
        }
    }
}
```

导航任务固定使用 `data.kind = 2`，任务参数放在 `data.params` 中。

字段	类型	说明
data.params.map_path	string	地图路径
data.params.need_locate	bool	可选，默认 `true`。导航起步前是否重新载图并定位：`true`=载图并等定位收敛（安全默认）；`false`=跳过，复用当前已加载地图的定位（免重载、保住位姿）。设 `false` 时须确保机器人已在该

		地图上定位好，否则起步守卫会判定位未就绪而失败。
data.params.map_type	number	导航地图类型：`1` =2D 栅格、`2` =3D（预留，当前暂不支持）、`3` =Route 路网
data.params.goal_type	number	导航目标类型：应与 `map_type` 一致；`1` =2D、`2` =3D（预留，当前暂不支持）、`3` =Route 路网
data.params.fence_path	string	电子围栏路径
data.params.goals	array	有序目标点数组，至少包含一个目标点；单点导航数组长度为 1，多点导航按数组顺序执行
data.params.goals[].position.x	number	目标点 X 坐标
data.params.goals[].position.y	number	目标点 Y 坐标
data.params.goals[].position.z	number	目标点 Z 坐标
data.params.goals[].orientation.x	number	目标点姿态四元数 X
data.params.goals[].orientation.y	number	目标点姿态四元数 Y
data.params.goals[].orientation.z	number	目标点姿态四元数 Z
data.params.goals[].orientation.w	number	目标点姿态四元数 W
data.params.speed_limit.vx	number	X 方向速度限制
data.params.speed_limit.vy	number	Y 方向速度限制
data.params.speed_limit.vz	number	Z 方向速度限制
data.params.goal_tolerance.x	number	X 方向到点容差
data.params.goal_tolerance.y	number	Y 方向到点容差
data.params.goal_tolerance.theta	number	航向角到点容差

> SDK 公共接口继续使用 `MapType::kGrid=0`、`MapType::kRoadNetwork=1`；SDK 在协议编码时分别转换为本节导航协议的 `1` 和 `3`。SDK 请求中的 `goalType=0` 表示跟随地图类型，编码后会自动得到匹配的 `goal_type`。
>
> 当前导航实现仅支持 `1`（2D 栅格）和 `3`（Route 路网）。`2`（3D）仅为协议预留值，暂不支持；下发 3D 导航任务会返回“不支持/未实现”错误，不会启动导航。

2.5.1.5 回充任务（无图对桩） `5000`

代码块

```
{
    "head": {
        "type": 5000,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "kind": 3,
        "name": "xxxxx",
        "schedule": 0,
        "execute_at": "1757310525610",
        "priority": 0,
        "record": false,
        "params": null
    }
}
```

2.5.1.6 出桩任务 `5000`

代码块

```
{
    "head": {
        "type": 5000,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "kind": 5,
        "name": "xxxxx",
        "schedule": 0,
        "execute_at": "1757310525610",
        "priority": 0,
        "record": false,
        "params": null
    }
}
```

2.5.2 取消任务 `5002/5003`

2.5.2.1 请求示例 `5002`

代码块

```
{
    "head": {
        "type": 5002,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "task_id": "xxxxxxx"
    }
}
```

2.5.2.2 响应示例 `5003`

代码块

```
{
    "head": {
        "type": 5003,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

2.5.3 暂停任务 `5004/5005`

2.5.3.1 请求示例 `5004`

代码块

```
{
    "head": {
        "type": 5004,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
```

```
    "data": {
        "task_id": "xxxxxxx"
    }
}
```

2.5.3.2 响应示例 `5005`

代码块

```
{
    "head": {
        "type": 5005,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

2.5.4 恢复任务 `5006/5007`

2.5.4.1 请求示例 `5006`

代码块

```
{
    "head": {
        "type": 5006,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "task_id": "xxxxxxx"
    }
}
```

2.5.4.2 响应示例 `5007`

代码块

```
{
```

```
    "head": {
        "type": 5007,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok"
    }
}
```

2.5.5 自主返航回充 `5014/5015`

用于指示机器人**导航至当前地图已设置的回充点位（协议 `4018`），并自主对桩充电**。与上文各条不同，本指令**不依附于任何任务、不携带 `task_id`**，可在任意时刻下发；服务端将新建一个『返回充电点』任务，并在响应中返回其 `task_id`。

> 与 `5000 kind=3` 回充任务的区别：`kind=3` 是**无图一键对桩**（在充电桩附近直接对桩，不导航、不需要地图）；`5014` 是**先导航到预设回充点、再对桩**，因此要求已载入地图且该地图设过回充点。
>
> 与巡检 `end_task=2` 的区别：`end_task=2` 是创建巡检任务时约定"跑完最后一个航点再返充"；`5014` 是任意时刻临时触发。二者复用同一套"导航到回充点 → 对桩"逻辑。

当前有任务在跑时的处理：

当前状态	处理
空闲	直接执行
巡检中	**先取消巡检**，等其停稳后自动执行；`task_id` 在响应里**立即**返回，期间可用 `5002` 取消本次返航
回充中 / 已在返回充电点途中	**幂等**：直接返回成功，`data.task_id` 为**既有任务**的 id，不新建、不打断
建图 / 导航 / 定位中	拒绝（`0x3649` 资源冲突）。不擅自打断其它作业——建图中途停止会丢失未保存的地图

2.5.5.1 请求示例 `5014`

代码块

```
{
    "head": {
```

```
        "type": 5014,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "map_id": "xxxxxxx"
    }
}
```

字段	类型	说明
data.map_id	string	**可选**，仅作校验。回充点是每张地图一份、坐标只在所属地图坐标系内成立，故服务端**总是使用机器人当前已载入的地图**；若传入的 `map_id` 不是当前地图则拒绝（`0x364A`），而不会改用该地图的回充点。缺省或空串即不校验。

2.5.5.2 响应示例 `5015`

代码块

```
{
    "head": {
        "type": 5015,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok",
        "task_id": "xxxxxxx"
    }
}
```

字段	类型	说明
data.task_id	string	本次「返回充电点」任务的 id。用 `5002` 可取消（**包括巡检尚未停稳、任务还在 `pending_task` 的阶段**）；执行过程经 `5009` 上报（`return_to_charge.navigating` →

		`return_to_charge.arrived` → `return_to_charge.recharging`）。

响应 `code=0` 仅表示"已受理并已建任务"，实际到点与对桩结果经 `5009` 体现。

常见拒绝原因：

错误码	含义
`0x3658`	当前未载入定位地图（尚未载图/定位），无从确定用哪张图的回充点
`0x3672`	当前地图未设置回充点（`4018` 未下发）
`0x364A`	传入的 `map_id` 不是当前已载入的地图
`0x3649`	有建图/导航/定位任务正在运行

2.5.6 任务控制汇总

取消、暂停、恢复通过 `data.task_id` 指定目标任务。`5012` 在巡检运行中通过该字段指定巡检任务；巡检结束后该字段可省略或填写旧巡检 ID，响应会返回独立返程的新任务 ID。

操作	请求 type	响应 type
取消任务	`5002`	`5003`
暂停任务	`5004`	`5005`
恢复任务	`5006`	`5007`
巡检返回起点	`5012`	`5013`

「返回充电点」`5014/5015` 不在此表：`5014` 不是对已有任务的控制，而是新建任务；响应 `5015` 会返回本次新建任务的 `task_id`，详见 §4.6。

字段	类型	说明
head.type	number	消息类型，见上方操作类型表
head.time	number	消息时间戳，单位毫秒
head.src	number	请求固定为 `1`，响应固定为 `2`
head.msg_id	number	消息 ID，用于请求与响应关联

data.task_id	string	任务 ID
data.code	number	响应码， `0` 表示成功，非 `0` 表示失败
data.msg	string	响应说明

2.5.6.1 即时响应、`5009` 与终态事件的关系

每条任务控制链路都是三层：请求的**即时响应**只表示受理，执行**过程**走 `5009` ，底层**终态**另有独立事件
（ `5012` → `5017` ，回充 / 出桩 → `5019` ）。用户端(APP) 以 `task_id` 关联三者。

触发操作	即时响应	`5009` 任务标识	典型事件序列
返回充电点 `5014`	`5015` ， `code=0` 表示已建任务，并返回新的 `task_id`	新建任务， `type=return_to_charge`	`return_to_charge.navigating` → `return_to_charge.recharging` → `5009` 终态 + `5019` 底层结果
创建回充任务 `5000 kind=3`	`5001` ，成功时返回 `task_id`	创建任务， `type=recharge`	`recharge.docking` → `recharge.charging` → `5009` 终态 + `5019` 底层结果
创建出桩任务 `5000 kind=5`	`5001` ，成功时返回 `task_id`	创建任务， `type=undock`	`undock.running` → `5009` 终态 + `5019` 底层结果

阶段对应的 `lifecycle` 、 `stage` 和错误信息以 §5 为准；报文格式见 §2.1.2（ `5017` ）与 §2.1.3（ `5019` ）。

当 `5014` 已进入对桩 / 充电阶段后被 `5000 kind=5` 接管，原 `5014` 任务以取消 / 被替代终态结束，新的 `type=undock` 任务使用新的 `task_id` 继续通过 `5009` 上报出桩过程。

2.6 任务状态推送 `5009`

`5009 V2` 是算法侧权威的**机器人任务快照**。算法侧按 `task.report_period_ms` 周期推送（默认 500ms）；当任务生命周期、阶段、暂停状态或离散事件发生变化时，立即补帧。每次仅发布一帧，其中同时包含当前任务、等待接管的任务与最近终态，用户端(APP) 无需再合并多条任务帧。

2.6.1 完整示例

代码块

```
{
    "head": {
        "type": 5009,
        "time": 1786344054741,
```

```
        "src": 2,
        "msg_id": 12347
    },
    "data": {
        "schema_version": 2,
        "boot_id": "81697d4e-b754-48c8-8bce-d6677eb4881e",
        "revision": 42,
        "reason": "event",
        "robot": {
            "task_state": "running",
            "loc_state": {
                "status": 100,
                "fresh": true,
                "age_ms": 86
            }
        },
        "current_task": {
            "task_id": "018f2f5d-1c20-7a5b-a801-123456789003",
            "name": "A区例行巡检",
            "type": "patrol",
            "lifecycle": "running",
            "stage": "patrol.point_arrived",
            "progress": 40.0,
            "timing": {
                "created_at_ms": 1786344000100,
                "started_at_ms": 1786344000120,
                "ended_at_ms": null,
                "duration_ms": 54421
            },
            "schedule": "immediate",
            "params": {
                "map_id": "aaa",
                "path_id": "bbb",
                "map_type": 0,
                "end_task": 0
            },
            "last_event": {
                "seq": 2,
                "type": "patrol.point_arrived",
                "occurred_at_ms": 1786344054741,
                "data": {
                    "index": 1,
                    "total": 5,
                    "name": "A区巡检点",
                    "kind": 1
                }
            },
```

```
            "stop_reason": null,
            "error": null,
            "detail": {
                "last_arrived_point": {
                    "index": 1,
                    "total": 5,
                    "name": "A区巡检点",
                    "kind": 1,
                    "arrived_at": "2026-08-10 14:40:54.741",
                    "arrived_at_ms": 1786344054741
                }
            }
        },
        "pending_task": null,
        "recent_terminal_tasks": []
    }
}
```

空闲时仍持续推送，`current_task` 和 `pending_task` 均为 `null`：

代码块

```
{
    "schema_version": 2,
    "boot_id": "81697d4e-b754-48c8-8bce-d6677eb4881e",
    "revision": 48,
    "reason": "heartbeat",
    "robot": {
        "task_state": "idle",
        "loc_state": { "status": 100, "fresh": true, "age_ms": 120 }
    },
    "current_task": null,
    "pending_task": null,
    "recent_terminal_tasks": []
}
```

2.6.2 顶层字段

字段	类型	说明
`data.schema_version`	number	固定为 `2`
`data.boot_id`	string	本次算法进程启动 ID；进程重启后变化。用户端 (APP) 应在变化时清空旧任务和事件游标

`data.snapshot_seq`	number	5009 完整快照序号；每生成一帧递增，同一 `boot_id` 内用于丢弃重复/乱序帧
`data.revision`	number	任务快照内容修订号；任务状态、事件或终态保留集合变化时递增，心跳期间可保持不变
`data.emitted_at_ms`	number	ALG 生成本帧完整快照的 Unix epoch 毫秒时间
`data.reason`	string	本帧原因： `heartbeat` / `state_changed` / `event` / `terminal` / `paused` / `resumed`
`data.robot.task_state`	string	当前任务占用状态： `idle` / `running` / `paused`
`data.robot.loc_state.status`	number	定位状态码，取值见 §5.7
`data.robot.loc_state.fresh`	boolean	最近定位状态是否在 3000ms 新鲜度窗口内
`data.robot.loc_state.age_ms`	number/null	距最近定位状态的毫秒数；从未收到时为 `null`
`data.current_task`	object/null	当前已启动并持有任务资源的任务；空闲时为 `null`
`data.pending_task`	object/null	已受理但等待旧任务停稳 / 释放资源的任务；无排队接管时为 `null`
`data.recent_terminal_tasks`	array	最近终态任务，最新在前；最多 3 条、保留约 30 秒，用于终态丢包补偿

2.6.3 任务对象

`current_task` 、 `pending_task` 与 `recent_terminal_tasks[]` 使用完全相同的结构：

字段	类型	说明
`task_id`	string	任务 ID；取消 / 暂停 / 恢复按此 ID 操作
`name`	string	创建任务时传入的任务名称
`type`	string	稳定任务类型，见 §5.4；不再暴露 `5000 kind`
`lifecycle`	string	稳定任务生命周期，见 §5.5；用户端(APP) 判断任务是否运行只看此字段
`stage`	string	稳定业务阶段，见 §5.6；不直接透出底层状态串
`progress`	number/null	`0` – `100` ；无进度语义时为 `null` ，字段不会缺失

`timing.created_at_ms`	number	任务创建时间，Unix epoch 毫秒
`timing.started_at_ms`	number/null	真实开始执行时间；待启动时为 `null`
`timing.ended_at_ms`	number/null	终态时间；未终态时为 `null`
`timing.duration_ms`	number	单调时钟计算的执行时长，避免系统时钟校准造成负值
`schedule`	string	`immediate` / `scheduled`；当前仅支持 `immediate`
`params`	object/null	创建任务的原始参数；按 `type` 解析
`active_operation`	object/null	当前业务动作；不暴露 mission 派生子任务 UUID。运行中至少含 `type`、`stage`、`label`；终态为 `null`
`termination`	object/null	终态信息；运行中为 `null`，终态含 `kind`、`source`、`code`、`message`、`replaced_by_task_id`
`last_event`	object/null	最近一个离散事件；事件发生后会在后续心跳持续保留，直到被下一事件覆盖
`stop_reason`	object/null	主动停止 / 被替代原因：`{code, message}`；不是故障
`error`	object/null	真正失败信息：`{code, domain, underlying_code, message}`
`detail`	object	类型专属运行快照；无内容时为空对象 `{}`

`last_event.seq` 在单个任务内从 `1` 起递增。用户端(APP) 须使用 `boot_id + snapshot_seq` 判断快照新旧，并使用 `boot_id + task_id + last_event.seq` 对离散事件去重；不应使用 `reason=event` 直接触发重复业务动作。

`active_operation` 示例：

代码块

```
{
    "type": "navigation",
    "stage": "patrol.navigating",
    "label": "前往巡检点 2"
}
```

`label` 仅用于展示，程序判断必须使用 `type` 和 `stage`。巡检的导航、定位、回充等内部子任务仍不单独上报，
其业务动作折算到父巡检任务；用户端(APP) 不应把 `active_operation` 当作可操作的子任务 ID。

`termination.source` 固定取自 `user` / `low_battery` / `superseded` / `safety` / `shutdown` / `system` / `unknown`。仅当 `stop_reason.code` 能够可靠识别来源时映射为具体值，否则映射为 `unknown`；
`replaced_by_task_id` 在当前无稳定关联时取 `null`。

2.6.4 type枚举

type	说明
`mapping`	建图任务（`5000 kind=1`）
`navigation`	导航任务（`5000 kind=2`）
`recharge`	回充任务（`5000 kind=3`）
`patrol`	巡检任务（`5000 kind=4`）
`undock`	出桩任务（`5000 kind=5`）
`return_to_charge`	`5014` 创建的返回充电点 mission

2.6.5 lifecycle枚举

lifecycle	说明
`pending`	已受理，等待旧任务停稳 / 资源释放
`starting`	正在启动能力
`running`	执行中
`paused`	底层已实际进入暂停态；仅请求受理不会提前变更
`cancelling`	正在安全停止 / 等待底盘收敛
`success`	成功终态
`failed`	失败终态，`error` 非空
`cancelled`	取消 / 主动让位 / 被替代终态；原因看 `stop_reason`

2.6.6 stage速查

类型	可能的 `stage`

`mapping`	`mapping.preparing` / `mapping.running` / `mapping.saving` / `mapping.success` / `mapping.error`
`navigation`	`navigation.localizing` / `navigation.navigating` / `navigation.goal_arrived` / `navigation.paused` / `navigation.succeeded` / `navigation.settling` / `navigation.error`
`patrol`	`patrol.preparing` / `patrol.undocking` / `patrol.localizing` / `patrol.capturing_origin` / `patrol.navigating` / `patrol.retrying` / `patrol.point_arrived` / `patrol.waiting` / `patrol.photographing` / `patrol.point_completed` / `patrol.stopping` / `patrol.stopped` / `patrol.returning` / `patrol.returning_to_charge` / `patrol.recharging` / `patrol.completed`
`recharge`	`recharge.docking` / `recharge.squatting` / `recharge.charging` / `recharge.fully_charged`
`undock`	`undock.running`
`return_to_charge`	`return_to_charge.preparing` / `return_to_charge.navigating` / `return_to_charge.arrived` / `return_to_charge.recharging` / `return_to_charge.stopping`

`stage` 用于程序判断和 UI 展示；算法侧新增阶段时只会新增字符串，不会改变已有字符串语义。

2.6.7 loc_state.status枚举

status	说明
0	Idle，空闲
1	Standby，就绪（可加载地图）
100	Locating，定位中（已收敛 / 正常）
120	MapLoading，加载地图中
121	Relocating，重定位中
122	Lost，定位丢失
203	Error，定位异常

210	Fail，定位失败
255	Unknown，尚未收到定位状态

2.6.8 普通导航到点事件

普通导航任务的 `params.goals` 按数组顺序执行。导航底层逐目标状态变为成功时，算法侧原子发布：

- `stage = navigation.goal_arrived`；
- `last_event.type = navigation.goal_arrived`，事件数据含 `index / total`；
- `detail.last_arrived_goal` 保存最近到达目标及确认时间，后续心跳继续携带。

示例：

代码块

```
{
    "stage": "navigation.goal_arrived",
    "last_event": {
        "seq": 3,
        "type": "navigation.goal_arrived",
        "data": { "index": 1, "total": 3 }
    },
    "detail": {
        "last_arrived_goal": {
            "index": 1,
            "total": 3,
            "arrived_at_ms": 1786344054741
        }
    }
}
```

`index` 从 `0` 开始。导航中途失败或取消不会产生目标到达事件；最终完成全部目标后，任务进入 `navigation.succeeded`。如果底层没有提供逐目标 `task_status_list` 成功状态，算法侧只能上报整组导航状态，不能推断中间目标是否到达。

2.6.10 用户端(APP) 接入规则

1. 每收到一帧 `5009`，整体替换本地机器人任务快照，不要把多帧按任务自行拼接。
2. 同一 `boot_id` 内按 `snapshot_seq` 丢弃重复或更旧快照；`boot_id` 变化时清空旧任务和事件游标。
3. 使用 `current_task.lifecycle` 判断运行 / 暂停 / 终态，使用 `stage` 判断业务阶段；`robot.task_state` 只表示任务占用状态，不等同于底盘运动、充电或定位状态。

4. `pending_task` 表示已受理但尚未驱动底盘，仍可使用其 `task_id` 调用 `5002` 取消。
5. 即时终态可能出现在 `current_task`；回收后仍会短暂存在于 `recent_terminal_tasks`。用户端(APP) 两处都应识别，并按 `task_id + lifecycle` 幂等处理。
6. `5017` 和 `5019` 仍是各自业务结果通知，不能由 `5009.last_event` 替代。

网关收到客户端 `1501` 上线通知后，ALG 会向该客户端定向推送一帧完整 `5009` 快照；客户端刷新或重连后不需要等待下一次周期心跳。

2.7 查询巡检记录 API `5010/5011` (暂不对外使用)

2.7.1 巡检记录查询请求 5010

代码块

```
{
    "head": {
        "type": 5010,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": null
}
```

当前查询请求不需要请求参数， `data` 固定为 `null`。

2.7.2 巡检记录查询响应 `5011`

代码块

```
{
    "head": {
        "type": 5011,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 0,
        "msg": "ok",
        "total": 2,
        "record_path_prefix": "/ota/alg_data",
        "skipped_count": 0,
            "items": [
```

```
                {
                    "task": {
                        "uuid": "019eb81c-3073-7c72-87fb-73f4d2ecbfdd",
                        "name": "哈哈哈222",
                        "status": "failed",
                        "created_at": "2026-06-12 03:15:15.701",
                        "execute_at": null,
                        "started_at": "2026-06-12 03:15:15.701",
                        "ended_at": "2026-06-12 03:15:15.705",
                        "duration": "0.004s",
                        "progress": 100.0,
                        "error_msg": "load localization map failed: map_id=2026_06_12_03_10_49, loc_status=Fail"
                    },
                    "properties": {
                        "schedule_type": "immediate",
                        "type": "patrol",
                        "map_id": "2026_06_12_03_10_49",
                        "path_id": "自定义路径1",
                        "periodic": false,
                        "map_type": 0,
                        "current_floor": 3,
                        "end_task": null
                    },
                    "phase": [
                        {
                            "index": 0,
                            "name": "load_loc_map:2026_06_12_03_10_49",
                            "kind": "load_localization_map",
                            "status": "failed",
                            "message": "load localization map failed: map_id=2026_06_12_03_10_49, loc_status=Fail",
                            "started_at": "2026-06-12 03:15:15.704",
                            "ended_at": "2026-06-12 03:15:15.704",
                            "duration": "0s"
                        },
                        {
                            "index": 1,
                            "name": "nav:1",
                            "kind": "navigate",
                            "status": "failed",
                            "message": "load localization map failed: map_id=2026_06_12_03_10_49, loc_status=Fail",
                            "started_at": "2026-06-12 03:15:15.705",
                            "ended_at": "2026-06-12 03:15:15.705",
                            "duration": "0s"
                        }
```

```
                    ]
                }
            ]
        }
    }
```

2.7.3 字段说明

字段	类型	说明
head.type	number	消息类型，固定为 `5011`，表示巡检记录查询响应
head.time	number	响应时间戳，单位毫秒
head.src	number	消息来源，算法侧固定为 `2`
head.msg_id	number	消息 ID，与查询请求中的 `msg_id` 对应
data.code	number	查询结果码，`0` 表示查询成功，非 `0` 表示查询失败
data.msg	string/null	查询结果说明
data.total	number	巡检记录总数
data.record_path_prefix	string	巡检记录根目录前缀；由 `patrol.record_dir` 配置推导，不含末尾的 `patrol_record` 目录
data.skipped_count	number	因文件读取或解析失败而跳过的记录数
data.items	array	巡检记录列表
data.items[].task	object	任务基础信息
data.items[].task.uuid	string	任务 UUID
data.items[].task.name	string	任务名称
data.items[].task.status	string	任务状态
data.items[].task.created_at	string	任务创建时间
data.items[].task.execute_at	string/null	预约执行时间
data.items[].task.started_at	string/null	任务开始时间

data.items[].task.ended_at	string/null	任务结束时间
data.items[].task.duration	string	任务耗时
data.items[].task.progress	number	任务进度，范围 `0` 到 `100`
data.items[].task.error_msg	string/null	任务错误信息
data.items[].task.stop_reason	string/null	任务被主动中断时的原因；低电中断时为低电量及阈值信息
data.items[].properties	object	巡检任务属性
data.items[].properties.schedule_type	string	调度类型：`immediate`（立即）/ `scheduled`（预约）
data.items[].properties.type	string	任务类型（巡检固定为 `patrol`）
data.items[].properties.map_id	string	地图 ID
data.items[].properties.path_id	string	路径 ID
data.items[].properties.periodic	boolean	是否周期执行
data.items[].properties.map_type	number	APP 巡检请求的地图类型：`0` 2D 栅格地图 / `1` 路网；`RAM_ENABLE_MAP_ROUTE_3D=true` 只覆盖 ROS 导航资源，不改写此请求字段
data.items[].properties.current_floor	number/null	巡检终态落盘时最新的导航楼层；进程启动后尚未收到 `NavigationState` 时为 `null`
data.items[].properties.end_task	string/null	任务结束后的动作：`null`（无）/ `return_to_start`（返回起点）/ `return_to_charge`（返回充电）
data.items[].phase	array	任务执行阶段列表
data.items[].phase[].index	number	阶段序号
data.items[].phase[].name	string	阶段名称
data.items[].phase[].kind	string	阶段类型
data.items[].phase[].point_index	number	可选，导航阶段对应的起始航点下标，0-based；非导航阶段不返回
data.items[].phase[].point_type	string	可选，`waypoint`（途径点）/ `workpoint`（作业点）

data.items[].phase[].pose	object	可选，导航阶段对应航点位姿，包含 `position` 和 `orientation`
data.items[].phase[].image_path	string/null	拍照阶段的图片路径；拍照成功且上装返回路径时为实际路径，否则为 `null`
data.items[].phase[].status	string	阶段状态
data.items[].phase[].message	string/null	阶段说明或错误信息
data.items[].phase[].started_at	string/null	阶段开始时间
data.items[].phase[].ended_at	string/null	阶段结束时间
data.items[].phase[].duration	string	阶段耗时

2.8 删除巡检记录 API `5020/5021`（暂不对外使用）

按 `task_id` 列表批量删除机器人上已落盘的巡检任务记录。删除后摘要索引同步重建，故经 HTTP 直取 `index.json` 的那条路径也立即一致。

2.8.1 删除巡检记录请求 `5020`

代码块

```
{
    "head": {
        "type": 5020,
        "time": 1757310525610,
        "src": 1,
        "msg_id": 12345
    },
    "data": {
        "task_id": ["3f2a1c8e-...-a1", "7b9d4e2f-...-c3"]
    }
}
```

字段	类型	必填	说明
head.type	number	是	消息类型，固定为 `5020`

data.task_id	string[]	条件必填	待删记录的任务 ID 数组，**不能为空**，单次最多 `20` 条、单个 ID 最长 `64` 字符（超过单次上限时应分批发送请求）。取值即 `5011` 的 `items[].task.uuid`。`all = true` 时**不得携带本字段**
data.all	boolean	否	`true` = **清空该机器人上的全部巡检记录**，此时忽略条数上限、且不得同时给 `task_id`。缺省 / `false` 时走按 ID 删除除， `task_id` 必填

两种模式二选一

代码块

```
{ "head": { "type": 5020, "time": 1757310525610, "src": 1, "msg_id": 12346 },
  "data": { "all": true } }
```

- **按 ID 删除**：`data.task_id` 给非空数组，不给 `all`（或给 `all: false`）。
- **清空全部记录**：`data.all` 显式给 `true`，不给 `task_id`。
- ⚠️ `all` 必须**显式**为 `true` 才会清空 —— 服务端**绝不**把"`task_id` 缺失或为空数组"解释成清空，那种请求一律 `code = 0x364A` 拒掉。这是为了让客户端拼错 ID 列表的 bug 不至于静默清库。
- ⚠️ 同时给 `all: true` 与非空 `task_id` 属意图矛盾，直接返回 `code = 0x364A`，**一条都不删**。

2.8.2 删除巡检记录响应 `5021`

代码块

```
{
    "head": {
        "type": 5021,
        "time": 1757310525610,
        "src": 2,
        "msg_id": 12345
    },
    "data": {
        "code": 14003,
        "msg": "some patrol records could not be deleted",
        "deleted": ["3f2a1c8e-...-a1", "7b9d4e2f-...-c3"],
        "failed": [{ "task_id": "5c8e2a1f-...-b7", "msg": "delete failed
(storage error)" }],
        "remaining_total": 48
```

```
14      }
15  }
```

字段	类型	说明
head.type	number	消息类型，固定为 `5021`，表示删除巡检记录响应
data.code	number	结果码。`0` = 全部达成；`14003`（`0x36B3`）= **部分条目未能删除**，其余已删，明细见 `failed`；其它非 0 = 整批未执行（见下）
data.msg	string	结果描述
data.deleted	string[]	目标已达成的任务 ID（真删掉的，以及记录目录内本就不存在的）
data.failed	object[]	本次**未能删除**的任务 ID 明细，元素为 `{task_id, msg}`；无失败时为空数组
data.failed[].task_id	string	删除失败的任务 ID
data.failed[].msg	string	可读的失败原因（服务端内部路径 / errno 等细节只记日志，不下发）
data.remaining_total	number	删除后剩余的巡检记录总条数

语义约定

- **幂等性：**请求中指定的任务 ID 即使已不存在，也计入 `deleted`。服务端以“记录已不存在”作为成功结果，无需区分记录已被本次请求删除还是此前已被其他客户端删除或自动清理。
- **部分成功：**单条记录因权限或 I/O 错误删除失败时，服务端继续处理其余记录，并将失败项写入 `failed`。处理完成后返回 `code = 14003`（`0x36B3`），同时返回完整的 `deleted`、`failed` 和 `remaining_total`。客户端应依据明细更新本地数据和界面。
- **部分成功的结果码：**部分记录删除失败时不得返回 `code = 0`。`code = 0` 仅表示请求涉及的全部记录均已达到“不存在”状态；非零结果码用于确保客户端不会将仍存在的记录误判为删除成功。
- **索引一致性：**无论本次处理是否存在失败项，服务端均重建摘要索引 `index.json`。索引内容应与本次删除结果保持一致，不保留已删除记录。

- **整批原子拒绝：**当 `task_id` 缺失、类型非法、为空数组、元素非字符串，或数组超过 `20` 条、单个 ID 超过 `64` 个字符时；以及任一 ID 等于 `.`、`..` 或保留名 `index`，包含路径分隔符（`/`、`\`）或 NUL 字节（`\0`）时，服务端返回 `code = 0x364A`，本批请求不删除任何记录。仅当 ID 完整等于上述保留值时才拒绝；例如 `a.b` 和 `index.json` 均为合法 ID。
- **列表刷新：**客户端可使用 `remaining_total` 更新列表总数，无需再次请求 `5010`。当 `failed` 非空时，建议重新请求 `5010`，以校准列表明细。

3. 本体 API

3.1 本体 WebSocket 接口

1. 必须先建立 `control`，并取得 `1201` 返回的 `session_id` 与 `bulkdata_bind_token`。
2. 建立 `bulkdata` 后，首个数据帧必须发送 `1202` 绑定请求。
3. `1202.data.session_id` 和 `1202.data.token` 必须分别使用 `1201` 返回的会话 ID 和绑定令牌。
4. 只有收到 `1203` 后，`bulkdata` 才视为绑定成功，可以继续传输业务数据。

1. 客户端连接 `ws://host:10010/control`。
2. 网关主动返回 `1201 kHandShakeAck`，客户端保存 `client_id`、`session_id` 和 `bulkdata_bind_token`。
3. 如需重新获取会话信息，客户端可在 `control` 上发送 `1200 kHandShake`，网关再次返回 `1201`。
4. 客户端连接 `ws://host:10010/bulkdata`。
5. 连接建立后的第一条消息发送 `1202 kBindBulkDataChannel`。
6. 网关返回 `1203 kBindBulkDataChannelAck`，确认绑定成功。
7. 绑定成功后，客户端再发送或接收需要走 `bulkdata` 的业务消息。

3.1.1 本体 WebSocket 接口详细定义（1200~1315、1501~1525）

`1200 kHandShake` — 握手请求（请求当前 session 信息）

- 方向：用户端(APP) / ALG / CLOUD / SDK → gateway
- 说明：请求当前 session 信息
- 触发：control 通道已建立后，如需再次获取当前 session 信息，可主动发送
- `data`：空 object 即可

示例：

代码块

```
{
  "head": {
    "type": 1200,
    "time": 1757310525615,
    "src": 1
  },
  "data": {}
}
```

`1201 kHandShakeAck` — 握手应答（control 建立后下发会话信息）

- 方向：gateway → 用户端(APP) / ALG / CLOUD / SDK
- 说明：control 建立后下发会话信息
- 触发：
  - control 连接建立成功后自动发送
  - 收到 `1200` 后再次发送

示例：

代码块

```
{
  "head": {
    "type": 1201,
    "time": 1757410000000,
    "src": 5
  },
  "data": {
    "client_id": 17,
    "session_id": "8f6c2a7b9d",
    "bulkdata_bind_token": "b45a11d3ef",
    "control_path": "/control",
    "bulkdata_path": "/bulkdata",
    "bind_timeout_ms": 30000,
    "type": "ZSL-2-Ultra"
  }
}
```

`data` 字段：

字段	含义	类型	备注
`client_id`	逻辑客户端 ID	uint32	gateway 分配

`session_id`	会话 ID	string	随机字符串
`bulkdata_bind_token`	bulkdata 绑定令牌	string	随机字符串
`control_path`	控制 path	string	默认 `/control`
`bulkdata_path`	大负载 path	string	默认 `/bulkdata`
`bind_timeout_ms`	绑定超时	uint64	默认 `30000`
`type`	设备型号	string	来自 `device_json_path` 指向的设备 JSON；读取失败或字段缺失时为空字符串

`1202 kBindBulkDataChannel` — bulkdata 绑定请求

- 方向：用户端(APP) / ALG / CLOUD → gateway
- 说明：bulkdata 绑定请求
- 触发：bulkdata 连接建立后第一条消息必须发送

示例：

代码块

```
{
  "head": {
    "type": 1202,
    "time": 1757410000000,
    "src": 1
  },
  "data": {
    "session_id": "8f6c2a7b9d",
    "token": "b45a11d3ef"
  }
}
```

`data` 字段：

字段	含义	类型	备注
`session_id`	会话 ID	string	来自 `1201`
`token`	绑定令牌	string	来自 `1201.bulkdata_bind_token`

`1203 kBindBulkDataChannelAck` — bulkdata 绑定成功应答

- 方向：gateway → 用户端(APP) / ALG / CLOUD
- 说明：bulkdata 绑定成功应答
- 触发：bulkdata 绑定成功后返回

示例：

```
代码块
1  {
2    "head": {
3      "type": 1203,
4      "time": 1757410000000,
5      "src": 5
6    },
7    "data": {
8      "client_id": 17,
9      "session_id": "8f6c2a7b9d",
10     "channel": "bulkdata"
11   }
12 }
```

`data` 字段：

字段	含义	类型	备注
`client_id`	逻辑客户端 ID	uint32	与 control 一致
`session_id`	会话 ID	string	与 `1201` 一致
`channel`	通道名称	string	固定 `"bulkdata"`

`1204 kTakeRoamerXControl` — 获取 RoamerX 控制权请求

- 方向：用户端(APP) / ALG / CLOUD → gateway
- 说明：获取 RoamerX 控制权请求
- 触发：control 通道建立后，客户端希望发送 `4000 ~ 5999` RoamerX 自由数据前发送
- `data`：空 object 即可

示例：

```
代码块
1  {
2    "head": {
```

```
    "type": 1204,
    "time": 1757410000000,
    "src": 1
  },
  "data": {}
}
```

`1205 kTakeRoamerXControlNotify` — 获取 RoamerX 控制权通知

- 方向：gateway → 用户端(APP) / ALG / CLOUD / SDK
- 说明：获取 RoamerX 控制权通知
- 触发：收到 `1204` 后返回；成功切换 owner 时广播给所有 control 通道客户端，失败时仅返回申请方

示例：

代码块

```
{
  "head": {
    "type": 1205,
    "time": 1757410000000,
    "src": 5
  },
  "data": {
    "status_code": 0,
    "message": "ok",
    "client_id": 17
  }
}
```

`data` 字段：

字段	含义	类型	备注
`status_code`	状态码	uint32	`0` 表示成功，非 `0` 表示失败
`message`	附加信息	string	成功时为 `ok`；失败时说明原因
`client_id`	当前 RoamerX 控制权 owner	uint32	gateway 分配的逻辑客户端 ID；失败时保持当前 owner

`1207 kModuleAvailabilityNotify` — 模块可用性主动通知

- 方向：gateway → 用户端(APP) / ALG / CLOUD / SDK
- 说明：gateway 缓存内部模块的最新可用状态，并按固定周期向所有 control 通道客户端广播
- 触发：gateway 默认每秒主动广播一次，与内部 `1525` 心跳的到达时刻解耦
- 超时：连续 3 秒未收到 SDK 的 `1525` 心跳时，gateway 将 `sdk_node.available` 置为 `false` 并继续定期广播
- 配置： `网关模块配置项` 控制广播周期， `sdk_heartbeat_timeout_ms` 控制 SDK 心跳超时
- `data.modules` 为可扩展模块列表；后续内部模块可在该数组中追加自己的状态项

示例：

代码块

```
{
  "head": {
    "type": 1207,
    "time": 1757410000000,
    "src": 5
  },
  "data": {
    "modules": [
      {
        "name": "sdk_node",
        "available": true
      }
    ]
  }
}
```

`data.modules` 元素字段：

字段	含义	类型	备注
`name`	模块名称	string	当前为 `sdk_node`
`available`	模块可用状态	bool	`false` 表示 SDK 尚未连接或正在断线重连；`true` 表示 SDK 已正常连接

`1299 kGatewayError` — **网关传输层错误通知**

- 方向：gateway → 用户端(APP) / ALG / CLOUD
- 说明：网关传输层错误通知
- 触发：transport 层错误

常见场景：

- path 不支持
- bulkdata 首包不是绑定请求
- `session_id` 缺失
- `token` 缺失
- token 校验失败
- 绑定超时
- session 已失效

示例：

代码块

```
{
  "head": {
    "type": 1299,
    "time": 1757410000000,
    "src": 5
  },
  "data": {
    "message": "bulkdata bind token invalid"
  }
}
```

`data` 字段：

字段	含义	类型	备注
`message`	错误描述	string	当前实现仅返回该字段

`1300 kHeartbeat` — 心跳请求

- 方向：外→本
- 说明：网关心跳请求；收到后刷新 control 会话保活并立即返回 `1301`
- `data`：空

示例：

代码块

```
{
  "head": {
```

```
    "type": 1300,
    "time": 1757410000000,
    "src": 1
  },
  "data": {}
}
```

`data` 字段：

无字段。

`1301 kHeartbeatNotify` — 心跳响应

- 方向：本→外
- 说明：网关心跳响应
- `data`：空
- 备注：`time` 为 gateway 发送响应时的本地时间，`src` 固定为 `5`

示例：

代码块

```
{
  "head": {
    "type": 1301,
    "time": 1757410000000,
    "src": 5
  },
  "data": {}
}
```

`data` 字段：

无字段。

`1302 kCommand` — 命令请求

- 方向：外→本
- 说明：命令请求
- `data`：`cmd`

示例：

代码块

```
{
```

```
  "head": {
    "type": 1302,
    "time": 1757410000000,
    "src": 1
  },
  "data": {
    "cmd": 1
  }
}
```

`data` 字段：

字段	含义	类型	备注
`cmd`	命令	uint32	具体枚举值由业务侧约定

`cmd` 枚举值：

值	说明
`0`	紧急停止
`1`	解除紧急停止
`10`	站立
`11`	匍匐
`12`	卧倒
`13`	锁定
`14`	爬高台
`15`	挡鼠板
`16`	部署姿态
`17`	换电姿态
`18`	过窄门
`19`	步态行走
`20`	扭一扭

21	陀螺
22	跳跃
23	双足站立
24	后空翻
50	头尾交换
60	通用模式
61	原地模式
62	导航模式
63	登阶模式
64	跟随模式
65	追踪模式
80	低速
81	中速
82	高速
83	导航速
90	同膝模式
91	内对膝模式
100	补光灯自动开启
101	补光灯自动关闭
102	前补光灯开
103	后补光灯开
104	前补光灯关
105	后补光灯关
110	避障开启
111	避障关闭

`1303 kCommandNotify` — 命令响应

- 方向：本→外
- 说明：命令响应
- `data`：`status_code`、`message`、`cmd`

示例：

代码块

```
{
  "head": {
    "type": 1303,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "status_code": 0,
    "message": "ok",
    "cmd": 1
  }
}
```

`data` 字段：

字段	含义	类型	备注
`status_code`	状态码	uint32	`0` 表示已收到或执行成功
`message`	附加信息	string	失败原因或补充说明
`cmd`	命令	uint32	回显收到的命令枚举值

`1304 kRemoteControl` — 遥控

- 方向：外→本
- 说明：遥控
- `data`：`lx`、`ly`、`rx`、`ry`、可选 `body.turn`、`body.high_low`

示例：

代码块

```
{
  "head": {
    "type": 1304,
    "time": 1757410000000,
```

```
    "src": 1
  },
  "data": {
    "lx": 0.1,
    "ly": 0.8,
    "rx": 0.0,
    "ry": 0.0,
    "turn": "none",
    "high_low": "none"
  }
}
```

`data` 字段：

字段	含义	类型	备注
`lx`	左右移动速度	float	`[-1, +1]`；运动模式下，正数左移，负数右移；原地模式下无作用
`ly`	前后移动速度	float	`[-1, +1]`；运动模式下，正数前进，负数后退；原地模式下无作用
`rx`	旋转速度/探头速度	float	`[-1, +1]`；运动模式下，正数左旋转，负数右旋转；原地模式下，正数左探头，负数右探头
`ry`	俯仰速度	float	`[-1, +1]`；运动模式下无作用；原地模式下，正数抬头，负数低头
`turn`	原地转身	string	`left` / `right` / `none`
`high_low`	身形高低	string	`up` / `down` / `none`

补充说明：

- 摇杆值非 0 时建议按 50Hz 发送
- 摇杆值全为 0 时建议按 5Hz 发送
- `turn` 和 `high_low` 为有效动作时，摇杆值建议发 0

`1310 kChargingPile` — **充电桩信息请求**

- 方向：外→本
- 说明：充电桩信息请求
- `data`：空

示例：

代码块

```
{
  "head": {
    "type": 1310,
    "time": 1757410000000,
    "src": 1
  },
  "data": {}
}
```

`data` 字段：

无字段。

`1311 kChargingPileNotify` — 充电桩信息通知

- 方向：本→外
- 说明：充电桩信息通知
- `data`：`status_code`、`message`、`sn`、`version`、`tag_id`

示例：

代码块

```
{
  "head": {
    "type": 1311,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "status_code": 0,
    "message": "ok",
    "sn": "CP-001",
    "version": "1.0.0",
    "tag_id": "TAG-1001"
  }
}
```

`data` 字段：

字段	含义	类型	备注
`status_code`	状态码	uint32	`0` 表示成功

`message`	附加信息	string	补充说明
`sn`	充电桩 SN	string	设备序列号
`version`	充电桩软件版本	string	版本号
`tag_id`	二维码 ID	string	充电桩标识

`1315 kRobotStatus` — 本体状态上报

- 方向：本→外
- 说明：本体状态上报
- 备注：1Hz 主动上报

示例：

代码块

```
{
  "head": {
    "src": 4,
    "time": 1786000946367,
    "type": 1315
  },
  "data": {
    "battery": {
      "current1": -1.5920000076293945,
      "current2": 0,
      "power1": 90,
      "power2": 0,
      "power_supply_status1": 2,
      "power_supply_status2": 0,
      "present1": true,
      "present2": false,
      "temperature1": 33,
      "temperature2": 0,
      "voltage1": 56.808998107910156,
      "voltage2": 0
    },
    "charging_pile_connect": false,
    "ctrl_source": 2,
    "estop": {
      "hardware": false,
      "software": false
    },
    "executor": "",
    "fill_light": {
```

```
      "auto_work": false,
      "back": false,
      "display_status": false,
      "front": false
    },
    "head_angle": 2.3519999980926514,
    "head_direction": "head",
    "knee": "",
    "mile_data": 0,
    "mode": "remote",
    "motion_status": "lie_down",
    "obstacle_avoidance": false,
    "silence_enable": false,
    "speed": {
      "level": "unknown",
      "x": 0.004999999888241291,
      "y": -0.0020000000949949026,
      "yaw": -0.0010000000474974513
    },
    "temp": 0,
    "uwb_connect": false
  }
}
```


`data` 字段

字段	含义	类型	备注
`head_angle`	头部角度	float	单位 rad
`head_direction`	头部方向	string	`head` / `tail` / `unknown`
`motion_status`	运动状态	string	枚举映射见下表
`mode`	机器模式	string	枚举映射见下表，来源于 SDK `MachineStatus`
`mile_data`	里程累积	float	单位 m
`battery`	电池信息	object	双电池结构，见下表
`speed`	速度信息	object	见下表
`fill_light`	补光灯状态	object	见下表

`estop`	急停状态	object	见下表
`ctrl_source`	控制源	uint32	`0=unknown, 1=APP, 2=SDK, 3=other`
`temp`	机体温度	float	—
`knee`	膝部状态	string	—
`obstacle_avoidance`	是否开启避障	bool	—
`executor`	执行器 / 执行来源	string	—
`charging_pile_connect`	是否连接充电桩	bool	—
`uwb_connect`	UWB 连接状态	bool	—
`silence_enable`	是否静默	bool	—

> 说明：`temp` / `knee` / `obstacle_avoidance` / `executor` / `charging_pile_connect` / `uwb_connect` / `silence_enable` 由 SDK 状态回调之外的模块补充填充，未填充时使用默认值。

`data.battery` 结构

字段	含义	类型	备注
`power1` / `power2`	电池1/2 电量	float	百分比 0-100
`present1` / `present2`	电池1/2 是否在位	bool	—
`voltage1` / `voltage2`	电池1/2 电压	float	单位 V
`current1` / `current2`	电池1/2 电流	float	单位 A
`temperature1` / `temperature2`	电池1/2 温度	float	单位 °C
`power_supply_status1` / `power_supply_status2`	电池1/2 电源状态	uint32	`0=UNKNOWN, 1=CHARGING, 2=DISCHARGING, 4=FULL`

`data.speed` 结构

字段	含义	类型	备注

`level`	速度等级	string	当前实现固定为 `unknown`
`x`	前后速度	float	单位 m/s
`y`	左右速度	float	单位 m/s
`yaw`	旋转角速度	float	单位 rad/s

`data.fill_light` 结构

字段	含义	类型	备注
`front`	前补光灯是否开启	bool	—
`back`	后补光灯是否开启	bool	—
`auto_work`	是否为自动灯光模式	bool	—
`display_status`	显示状态	bool	—

`data.estop` 结构

字段	含义	类型	备注
`software`	软件急停是否触发	bool	—
`hardware`	硬件急停是否触发	bool	—

`motion_status` 枚举映射（SDK `MotionStatus`）

SDK 枚举	字符串值	含义
`MOTION_STATUS_UNKNOWN`	`unknown`	未知
`MOTION_STATUS_STAND_UP`	`stand_up`	站立中
`MOTION_STATUS_WALK`	`walk`	行走
`MOTION_STATUS_BALANCE_STAND`	`balance_stand`	平衡站立
`MOTION_STATUS_LIE_DOWN`	`lie_down`	卧倒
`MOTION_STATUS_CRAWL`	`crawl`	匍匐
`MOTION_STATUS_CRAWL_WALK`	`crawl_walk`	匍匐行走

`MOTION_STATUS_LOCKED`	`locked`	锁定
`MOTION_STATUS_CLIMB`	`climb`	爬高台
`MOTION_STATUS_STAIR`	`stair`	登阶
`MOTION_STATUS_SLIM`	`slim`	矮身
`MOTION_STATUS_GAIT`	`gait`	步态
`MOTION_STATUS_DSB`	`dsb`	DSB
`MOTION_STATUS_POS_CONTROL`	`pos_control`	位控姿态
`MOTION_STATUS_SK_WALK`	`sk_walk`	同膝行走

`mode` 枚举映射（SDK `MachineStatus`）

SDK 枚举	字符串值	含义
`UNKNOWN`	`unknown`	未知
`IDLE`	`idle`	空闲
`REMOTE`	`remote`	远程控制
`OTA`	`ota`	OTA 升级
`RECHARGE`	`recharge`	回充
`MAPPING`	`mapping`	建图
`NAVIGATION`	`navigation`	导航
`SAFETY`	`safety`	安全保护
`SELFTEST`	`selftest`	自检
`SOFT_SHUTDOWN`	`soft_shutdown`	软关机
`SILENCE`	`silence`	静默
`FOLLOW`	`follow`	跟随
`TRACK`	`track`	追踪
`UNDOCK`	`undock`	离桩

`1317 kFaultStatus` — 故障状态上报

- 方向：本→外
- 说明：故障状态上报
- `data`：`faults[]`

示例：

```
代码块
1  {
2    "head": {
3      "type": 1317,
4      "time": 1757410000000,
5      "src": 4
6    },
7    "data": {
8      "faults": [
9        {
10         "module": "mc",
11         "submodule": "imu",
12         "fault": "IMU/ConnectError",
13         "level": 1,
14         "stamp": 1763377714
15       }
16     ]
17   }
18 }
```

`data` 字段：

字段	含义	类型	备注
`faults`	故障列表	array	每一项为一个故障对象
`faults[].module`	模块名	string	如 `mc`
`faults[].submodule`	子模块名	string	如 `imu`
`faults[].fault`	故障信息	string	如 `IMU/ConnectError`
`faults[].level`	故障等级	uint8	`0=OK,1=WARN,2=ERROR,3=FATAL_ERROR`
`faults[].stamp`	故障时间	uint32	秒级时间戳

`1319 kDiagnosis` — 诊断上报

- 方向：本→外

- 说明：诊断上报
- `data`：base64 编码诊断数据

示例：

```
{
  "head": {
    "type": 1319,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "payload": "ZGlhZ25vc2lzX2RhdGE="
  }
}
```

`data` 字段：

字段	含义	类型	备注
`payload`	诊断数据	string	base64 编码字符串，旧协议中说明可能为压缩后的诊断 JSON

`1323 kLightSensor` — 光传感器上报

- 方向：本→外
- 说明：光传感器上报
- `data`：如 `lux`

示例：

```
{
  "head": {
    "type": 1323,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "lux": 168.5
  }
```

```
10  }
```

`data` 字段：

字段	含义	类型	备注
`lux`	光照强度	float	旧协议与当前 sample 均使用该字段

`1325 kMotionControl` — 运控数据上报

- 方向：本→外
- 说明：运控数据上报
- 备注：运控数据

示例：

代码块

```
{
  "head": {
    "type": 1325,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "acc": [-0.017, -0.028, 0.020],
    "gyro": [-0.017, -0.028, 0.020],
    "omega_body": [-0.017, -0.028, 0.020],
    "omega_world": [-0.018, 0.028, 0.019],
    "position": [0.018, 0.007, 0.085],
    "quat": [-0.528, -0.004, 0.010, 0.848],
    "rpy": [0.023, -0.003, -2.027],
    "time_stamp": 1761340715651843436,
    "v_body": [-0.003, -0.017, -0.680],
    "v_world": [-0.001, 0.002, -0.680]
  }
}
```

`data` 字段：

字段	含义	类型	备注
`acc`	加速度	array	常见为三轴数组

`gyro`	陀螺仪	array	常见为三轴数组
`omega_body`	本体系角速度	array	常见为三轴数组
`omega_world`	世界系角速度	array	常见为三轴数组
`position`	位置	array	常见为 `[x,y,z]`
`quat`	四元数	array	常见为 `[x,y,z,w]`
`rpy`	欧拉角	array	常见为 `[roll,pitch,yaw]`
`time_stamp`	原始时间戳	uint64	旧协议 sample 中存在
`v_body`	本体系速度	array	常见为三轴数组
`v_world`	世界系速度	array	常见为三轴数组

说明：

- 旧协议字段较多，字段集合以实际下发内容为准

`1327 kSpeedReport` — 速度信息上报

- 方向：本→外
- 说明：速度信息上报
- 备注：速度信息

示例：

代码块

```
{
  "head": {
    "type": 1327,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "x": 0.3,
    "y": 0.0,
    "yaw": 0.1
  }
}
```

`data` 字段：

字段	含义	类型	备注
`x`	X 方向速度	float	旧协议 sample 使用
`y`	Y 方向速度	float	旧协议 sample 使用
`yaw`	偏航角速度	float	旧协议 sample 使用

`1329 kJointState` — 关节状态上报

1501 kOnlineNotify — 客户端上线通知

方向：网关 → 外部端。客户端建立会话后推送。

字段	类型	说明
`data.client_type`	uint32	客户端类型
`data.client_id`	uint32	网关分配的逻辑客户端 ID

1503 kOfflineNotify — 客户端下线通知

方向：网关 → 外部端。客户端会话断开后推送。

字段	类型	说明
`data.client_type`	uint32	客户端类型
`data.client_id`	uint32	下线客户端 ID

1521 kOnlineClientNotify — 在线客户端信息通知

方向：网关 → 外部端。在线客户端列表发生变化时推送。

`data` 为在线客户端 `client_id` 数组，例如 `[17, 18]`。

1523 kRoamerXControlNotify — RoamerX 控制权变更通知

方向：网关 → 外部端。收到 `1204` 后返回；控制权切换成功时广播。

字段	类型	说明
`data.no_control_duration_ms`	uint64	无控制权持续时间；控制权切换通知可为空对象
`client_id`	uint32	当前控制权 owner；无 owner 时为 `UINT32_MAX`

- 方向：本→外
- 说明：关节状态上报
- 备注：关节状态

示例：

代码块

```
{
  "head": {
    "type": 1329,
    "time": 1757410000000,
    "src": 4
  },
  "data": {
    "joints": [
      {
        "name": "front_left_hip",
        "position": 0.12
      }
    ]
  }
}
```

`data` 字段：

字段	含义	类型	备注
`joints`	关节列表	array	当前示例采用该结构
`joints[].name`	关节名称	string	如 `front_left_hip`
`joints[].position`	关节位置	float	角度或位置值

说明：

- 旧协议仅定义 JointState 上报，未提供字段明细；此处根据当前示例补充字段结构

`1525 kSdkHeartbeat` — **SDK 节点可用性心跳**

- 方向：SDK → gateway

`4000 ~ 5999` — **自由数据透传区间**

- 方向：双向
- 说明：自由数据透传区间

- 约束：客户端必须先通过 `1204 kTakeRoamerXControl` 获取 RoamerX 控制权；默认 owner 为 `UINT32_MAX`，此时 gateway 不转发任何非白名单 `4000 ~ 5999` 消息；owner 存在时，仅 owner client 的非白名单 `4000 ~ 5999` 消息会继续处理
- 白名单：`ws_server.roamerx.whitelist_types` 中配置的 type 即使没有 RoamerX 控制权也会继续处理，默认配置为 `4012 / 4022 / 5010`

本体接口通过 RobotAlgGateway 的 WebSocket `control` / `bulkdata` 通道承载。接口沿用统一消息结构：`head.type` 表示消息类型，`head.time` 为毫秒时间戳，`head.src` 标识来源，`head.msg_id` 用于请求与响应关联，`data` 为业务数据。

3.2 本体接口接入建议

1. 连接阶段先完成 1201 握手和 1202/1203 通道绑定，再发送本体控制请求或接收状态推送。
2. 控制类请求使用 `1302`、`1304`、`1310`；响应类消息以 `1303`、`1311` 的 `status_code` 和 `message` 判断受理结果。
3. 状态页以 1315 为主，故障页以 1317 为主，诊断工具接收 1319；地图页面分别处理 1401/1403/1405/1407。
4. 状态推送与算法任务 `5009` 分开存储和展示；`5009` 表示算法任务状态，`1315` 表示本体状态，不能相互覆盖。
5. 断线重连后重新处理 `1207`、`1521`、`1525`，并用消息时间戳避免旧状态覆盖新状态。

4. 文件服务

文件服务采用 HTTP 协议，端口为12000。
L2默认文件根路径为 `/userdata/alg_data`（M1 为 /ota/alg_data）。

HTTP 文件下载示例（以 L2 为例）：

HTTP 文件下载

```
curl -fL -o ./example.zip "http://192.168.168.100:12000/userdata/alg_data/example.zip
```